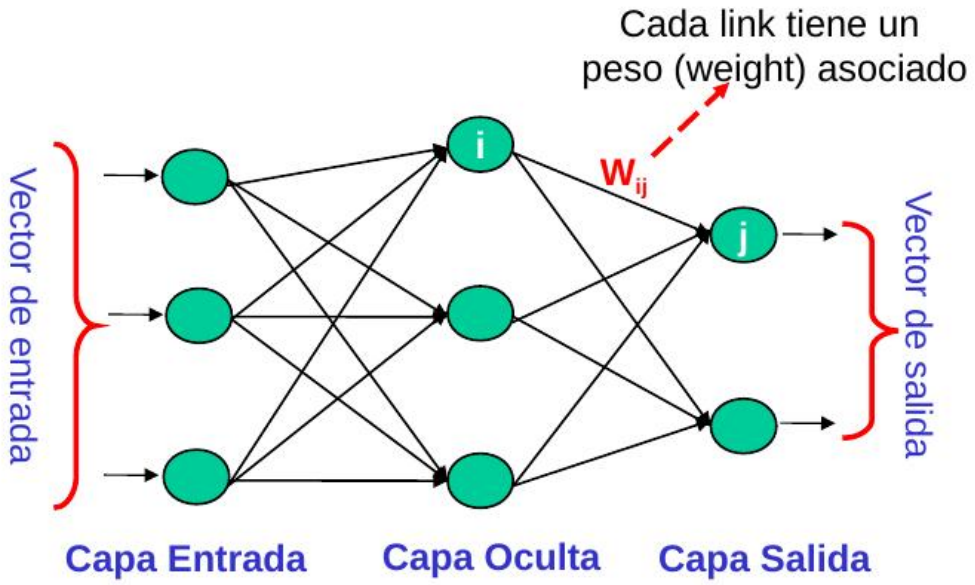
Pontificia Universidad Católica de Chile
IA Lab

Funciones de Activación

Gabriel Sepúlveda V.

Departamento de Ciencias de la Computación, PUC

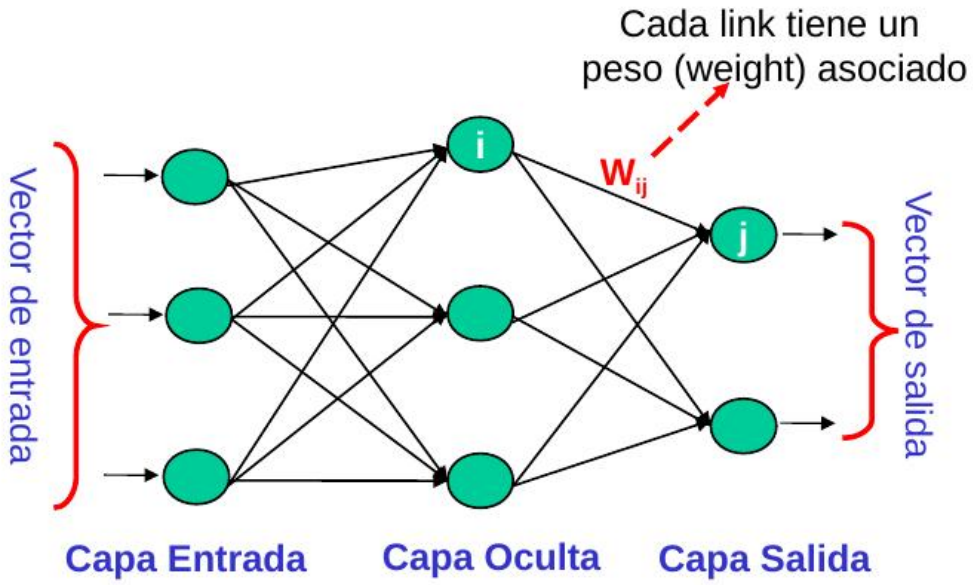
Redes Neuronales Artificiales (ANN)



La arquitectura de una red neuronal es definida por:

1. Estructura: número de capas, número de neuronas por capas
2. Método para cálculo de pesos W_{ij}, los cuales interconectan la neurona *i* con la neurona *j*
3. Función de Activación: modula la señal enviada desde una neurona a la siguiente

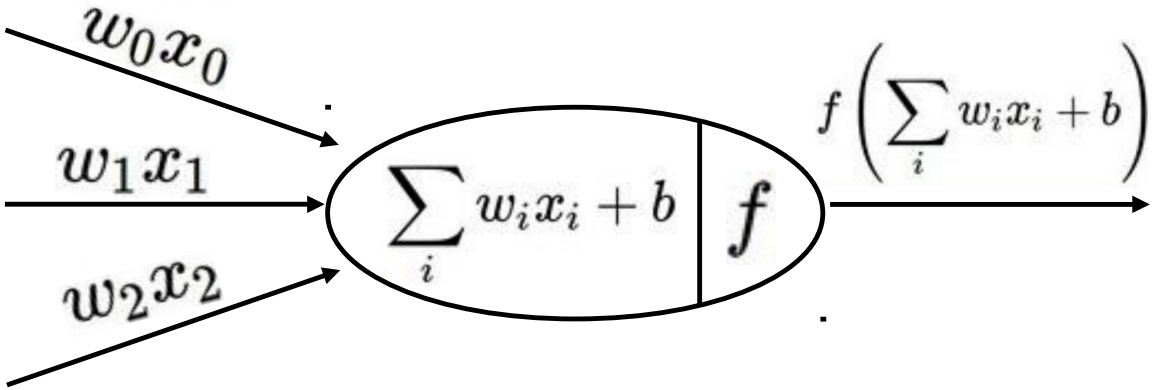
Redes Neuronales Artificiales (ANN)



La arquitectura de una red neuronal es definida por:

1. Estructura: número de capas, número de neuronas por capas
2. Método para cálculo de pesos W_{ij}, los cuales interconectan la neurona *i* con la neurona *j*
3. Función de Activación: modula la señal enviada desde una neurona a la siguiente

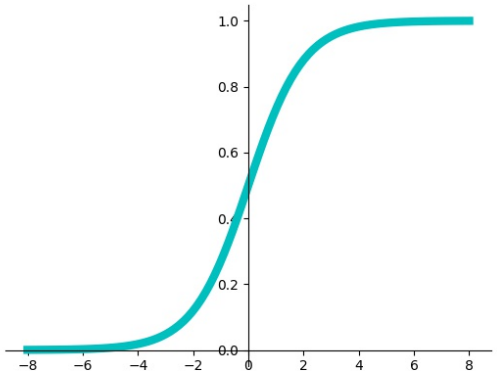
Perceptrón



¿Qué función de activación debemos usar para un perceptrón multicapa (MLPs)?

Funciones de Activación: Sigmoide

Función Sigmoide: $\sigma(x) = \frac{1}{1+e^{-x}}$



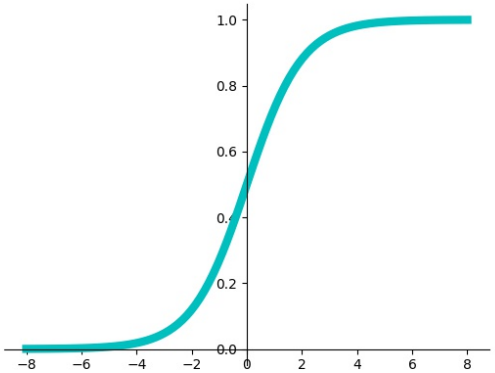
Buenas propiedades:

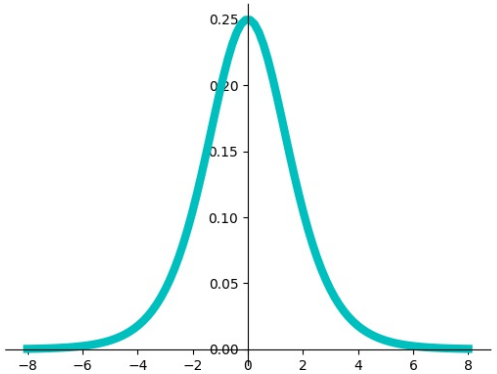
- Suave y derivable.
- Lleva salida de la red a valores extremos (0 y 1), lo que es útil para clasificación

Función Sigmoide:

$\sigma(x) = \frac{1}{1+e^{-x}}$

$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1 - \sigma(x))$





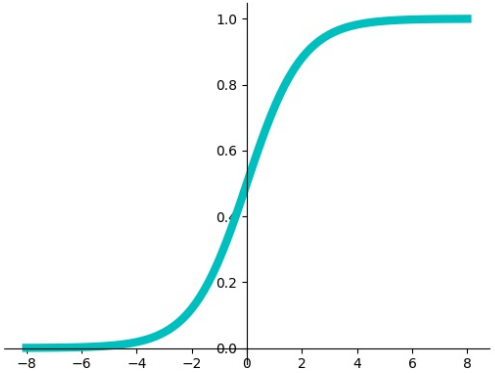
Mala propiedad 1:

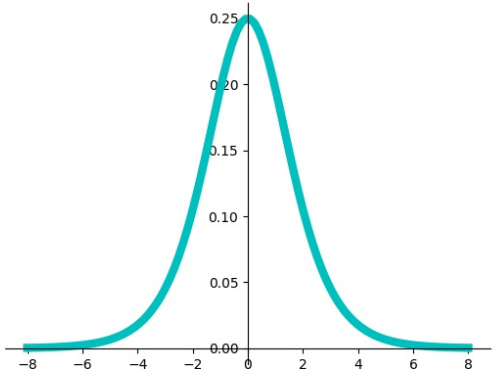
- Neuronas saturadas tienen gradiente cercano a cero.
- Recordemos actualización de pesos: $w_i^{new} = w_i^{old} - \eta \frac{\partial E}{\partial w_i}$
- ¿Algún problema?

Funciones de Activación: Sigmoide

$\sigma(x) = \frac{1}{1+e^{-x}}$

$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1 - \sigma(x))$



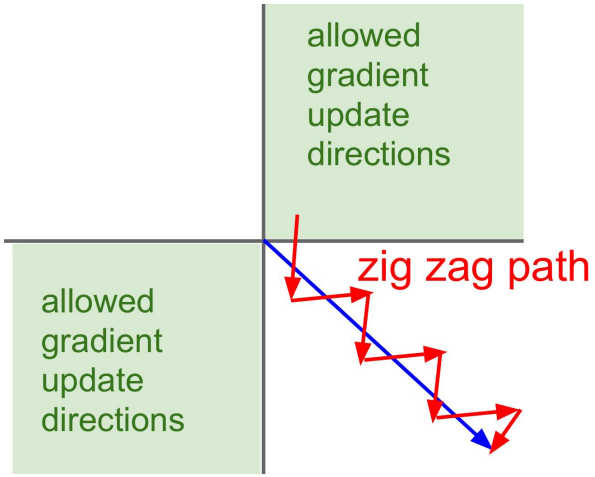


Mala propiedad 1:

- Neuronas saturadas tienen gradiente cercano a cero.
- Recordemos actualización de pesos: $w_i^{new} = w_i^{old} - \eta \frac{\partial E}{\partial w_i}$
- ¿Algún problema?
  - Convergencia lenta.
  - En deep learning, esto causa un problema conocido como *vanishing gradient*. Éste es un gran problema para redes recurrentes (RNNs), como veremos más adelante en el curso.

Mala propiedad 2:

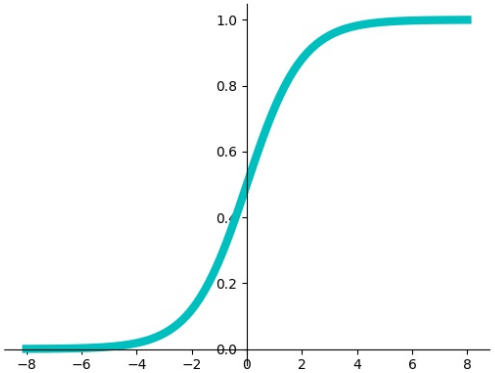
- Sigmoid outputs are not zero-centered.
- Debido a que la función no está centrada en cero, sus gradientes son todos positivos o todos negativos en el paso de actualización
- Lo anterior produce un efecto de zig-zag en la convergencia:

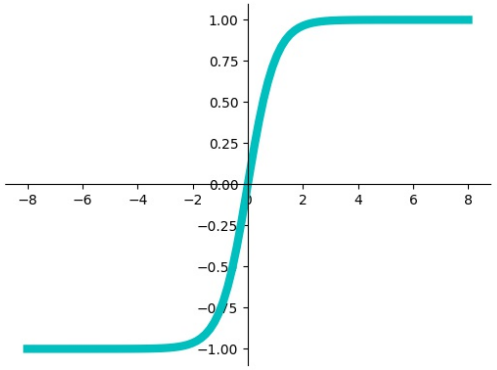


Función Tanh:

$\sigma(x) = \frac{1}{1+e^{-x}}$

$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}} = 2\sigma(2x) - 1$



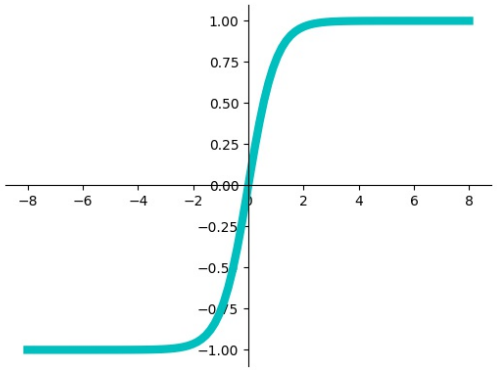


Buenas propiedades:

- Suave y derivable.
- Lleva salida de la red a valores extremos (0 y 1), lo que es útil para clasificación
- Centrada en cero.

Funciones de Activación: Tangente Hiperbólica

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$

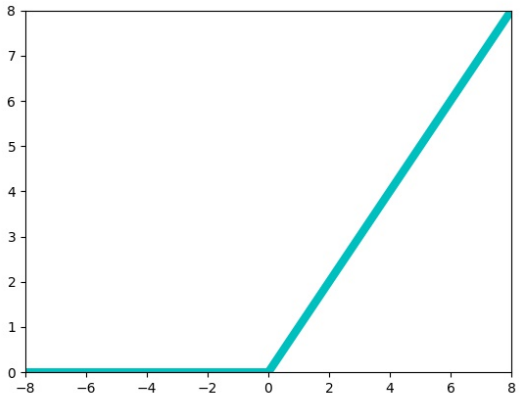


Malas propiedades:

- Neuronas en saturación tienen gradiente cercano a cero.
- Convergencia lenta y problema de *vanishing gradient*.
- Extremadamente sensible para valores de salida cercanos a cero.

Funciones de Activación: ReLU

Rectified Linear Unit: **ReLU**$(x) = max(0, x)$

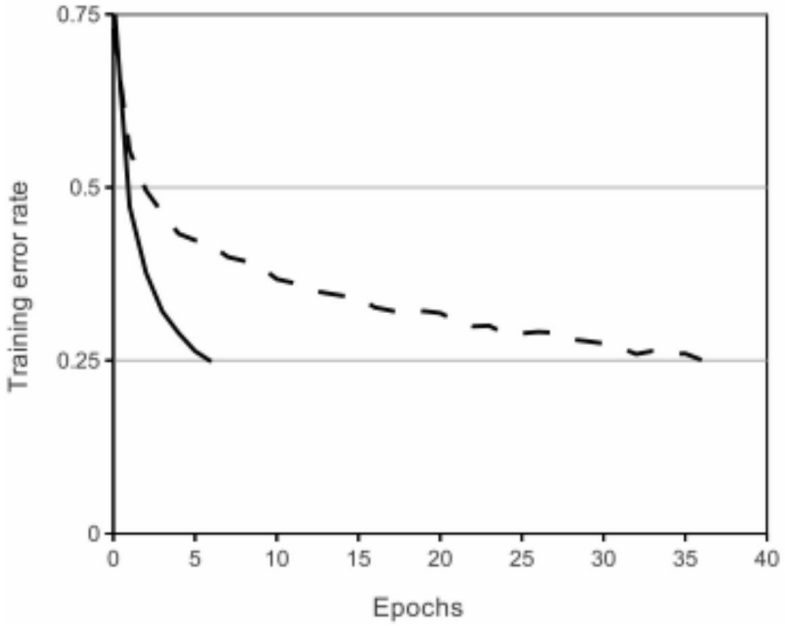


Buenas propiedades:

- No se satura para entradas positivas.
- Convergencia más rápida que usando función sigmoide (ej. 10x).
- Eficiente computacionalmente debido a que se calcula a través de una función lineal

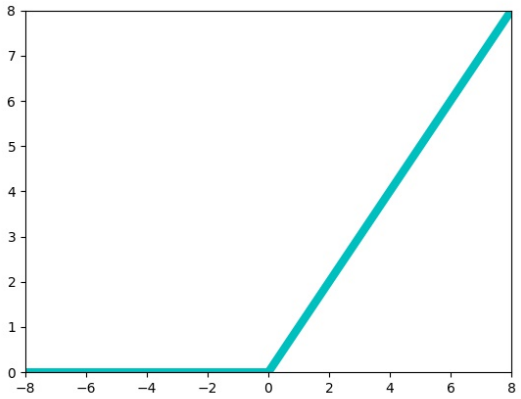
ReLU logra velocidad de convergencia mayor que la sigmoide

- Ejemplo (Krizhevsky et al., 2012): se entrena una CNN de 4 capas sobre CIFAR-10 (dataset de reconocimiento de objetos). Usando ReLUs (línea continua) se alcanza una tasa de error de 0.25 seis veces más rápido que la misma red utilizando $tanh(x)$ (línea punteada).



Funciones de Activación: ReLU

Rectified Linear Unit: **ReLU**$(x) = max(0, x)$

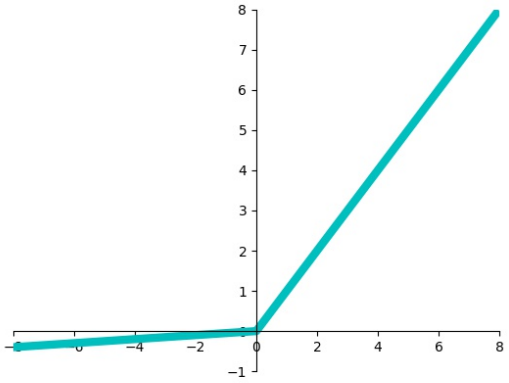


Malas propiedades:

1. No es derivable en cero.
2. Se satura para entradas negativas.
3. No es útil para problemas de clasificación.

Leaky/Parametric Rectified Linear Unit:

$$\textbf{PReLU}(x) = max(\alpha x, x)$$

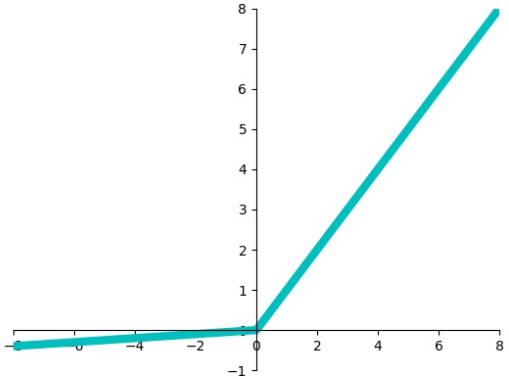


Buenas propiedades:

- No se satura (neuronas no dejan de aprender).
- Convergencia más rápida que usando función sigmoide (ej. 10x).
- Eficiente computacionalmente debido a que se calcula a través de funciones lineales.

Leaky/Parametric Rectified Linear Unit:

$$\textbf{PReLU}(x) = max(\alpha x, x)$$



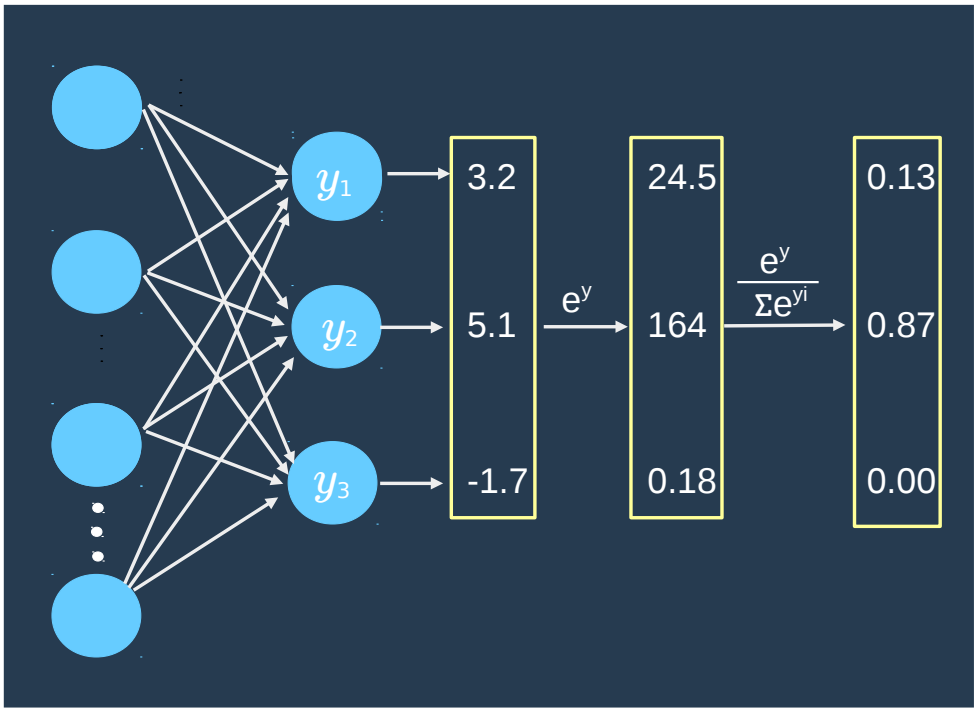
Bad properties:

- No es útil para problemas de clasificación.

Funciones de Activación: SoftMax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_i e^{z_i}}$$

- La función Softmax es usada normalmente en capas de salida
- Permite convertir salida de neuronas en valores que pueden ser interpretados como probabilidad.

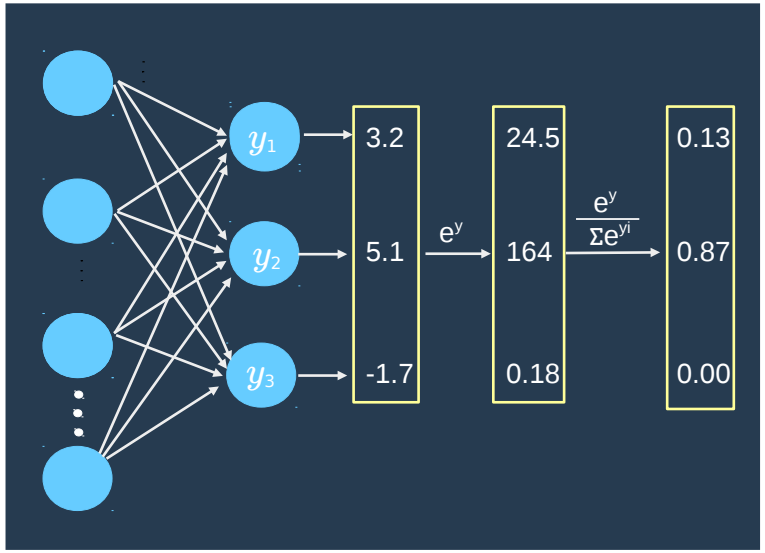


Funciones de Activación: SoftMax

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_i e^{z_i}}$$

Sofmax es más que una simple normalización

- Es derivable.
- Funciona como un "*maximizador suave*", amplificando los valores más altos y atenuando valores bajos. Incentiva competencia entre salidas (*the winner takes it all*).



Pontificia Universidad Católica de Chile
IA Lab

Funciones de Activación

Gabriel Sepúlveda V.

Departamento de Ciencias de la Computación, PUC